

Processo di Produzione del Software

Riassunto dei Concetti Fondamentali

1. Introduzione e Crisi del Software

- Il processo di produzione del software organizza il ciclo di vita del prodotto definendo attività, ordine di esecuzione e relazioni, con l'obiettivo di garantire standardizzazione, predicibilità, produttività, qualità e controllo dei costi.
- Agli albori, il modello utilizzato era il **Code and Fix** (scrivi il codice e correggi). Con la crescente complessità delle applicazioni, l'assenza di documentazione e la difficoltà di gestire le modifiche, questo approccio si rivelò un fallimento totale, portando alla cosiddetta **Crisi del Software** e alla nascita dell'Ingegneria del Software.
- L'adozione di un modello di processo esplicito serve ad evitare lo sviluppo "a scatola nera" (black-box), dove i progressi sono invisibili fino al rilascio finale, riducendo rischi e costi.

2. Le Attività Principali dello Sviluppo

Indipendentemente dal modello di processo adottato, vi sono delle attività fondamentali e universali:

- **Studio di fattibilità:** Valutazione iniziale per decidere se procedere con lo sviluppo, analizzando soluzioni alternative, costi e benefici.
- **Analisi e specifica dei requisiti:** Serve a capire cosa deve fare il sistema (non il *come*). I requisiti si dividono in funzionali, non funzionali (prestazioni, portabilità, ecc.) e di processo. Possono essere classificati in priorità: *MUST* (fondamentali), *SHOULD* (desiderabili) e *MAY* (facoltativi). Il risultato è il Documento di Specifica.
- **Definizione dell'architettura:** Produce la specifica di progetto, descrivendo i componenti del sistema e le loro interfacce.
- **Produzione di codice e test dei moduli:** Scrittura del codice seguendo gli standard aziendali e testing dei singoli moduli (white-box, black-box).
- **Integrazione e test del sistema:** Assemblaggio dei moduli e test globale dell'applicazione (Alpha test).
- **Rilascio, installazione e manutenzione:** Consegna (inizialmente come Beta test) e successiva manutenzione. Quest'ultima rappresenta la fase più costosa (60% del totale dei costi).

3. I Modelli di Processo

Il Modello a Cascata (Waterfall)

- Modello sequenziale basato sulla produzione intensiva di documentazione tra una fase e l'altra.
- **Vantaggi:** Impone disciplina e pianificazione.
- **Svantaggi:** Troppo rigido, lineare e monolitico. Presuppone (erroneamente) che i requisiti possano essere "congelati" all'inizio e non gestisce bene i feedback durante lo sviluppo.

Modelli Evolutivi e Incrementali

- Si basano sul principio del prototipo (spesso "throw-away", da buttare).
- Il software viene rilasciato in versioni incrementali (rilasci funzionali parziali e utilizzabili). Questo permette di misurare subito il valore aggiunto per il cliente e di aggiustare la rotta tempestivamente.

Il Modello a Spirale (Boehm)

- Modello ciclico fortemente basato sull'**analisi dei rischi**.
- Ogni ciclo (quadrante) prevede: 1) Individuazione degli obiettivi; 2) Analisi dei rischi e valutazione delle alternative; 3) Sviluppo e test (prototipazione); 4) Pianificazione della fase successiva. Il raggio della spirale rappresenta il costo accumulato.

4. Approcci Alternativi e Casi di Studio

- **Microsoft (Synchronize and stabilize):** Team paralleli, specifiche revisionate continuamente (fino al 30%), integrazione e compilazione quotidiana del codice (daily build) e rilascio a milestone prestabilite per stabilizzare il prodotto.
- **Open-Source:** Sviluppo distribuito tramite Internet. Si basa sulla libertà di modifica, sulla licenza *copyleft* e su una struttura gerarchica formata da un nucleo ristretto di sviluppatori principali e una vasta platea di contributori.

5. Unified Software Development Process (UP / RUP)

- Metodologia basata sul linguaggio **UML** (Unified Modeling Language), di natura iterativa e incrementale.
- Ogni macro-ciclo è suddiviso in quattro fasi, ognuna terminante con una *milestone*:
 1. **Concezione:** Analisi di fattibilità.
 2. **Elaborazione:** Identificazione dei casi d'uso e definizione dell'architettura (baseline).
 3. **Costruzione:** Sviluppo del codice per ottenere una versione rilasciabile.
 4. **Transizione:** Distribuzione, beta testing e formazione degli utenti.
- **Best practices del RUP:** Sviluppo iterativo, gestione rigorosa dei requisiti, uso di architetture a componenti, modellazione visuale (UML), controllo della qualità e gestione dei cambiamenti.